

Boldr Customer Intelligence Engine

A complete walkthrough for someone who has never seen this project

TL;DR — A customer service inbox that doesn't just *answer* messages, it *learns* from them. Every email from a Boldr Supply Co. customer becomes three things at once: a drafted reply, a knowledge-base improvement, and a marketing signal. Built for the **Echelon 2026 AI Workflow Competition**. Live demo: <https://boldr-intel.vercel.app>.

1. The problem in one paragraph

Boldr Supply Co. is a small Singapore-based titanium watch micro-brand. Their customer service inbox is full of repetitive but high-stakes questions: *Is the strap BPA-free? Can I engrave Chinese characters? Will it survive an MRI?* A three-person team can't write thoughtful replies to all of them, look up the right rate-card price, and *also* notice that they're getting asked the same novel question for the third time this month. So opportunities — a great FAQ entry, a marketing insight, a campaign idea — slip through the cracks.

This project automates the boring 80% (the lookups, the drafting) and surfaces the strategic 20% (what customers are asking that your product pages don't answer).

2. What the system does, end-to-end

Imagine a single customer message arrives:

"Hi, I have a nickel allergy. Can you confirm the buckle on the leather strap is nickel-free? I had a reaction with another brand."

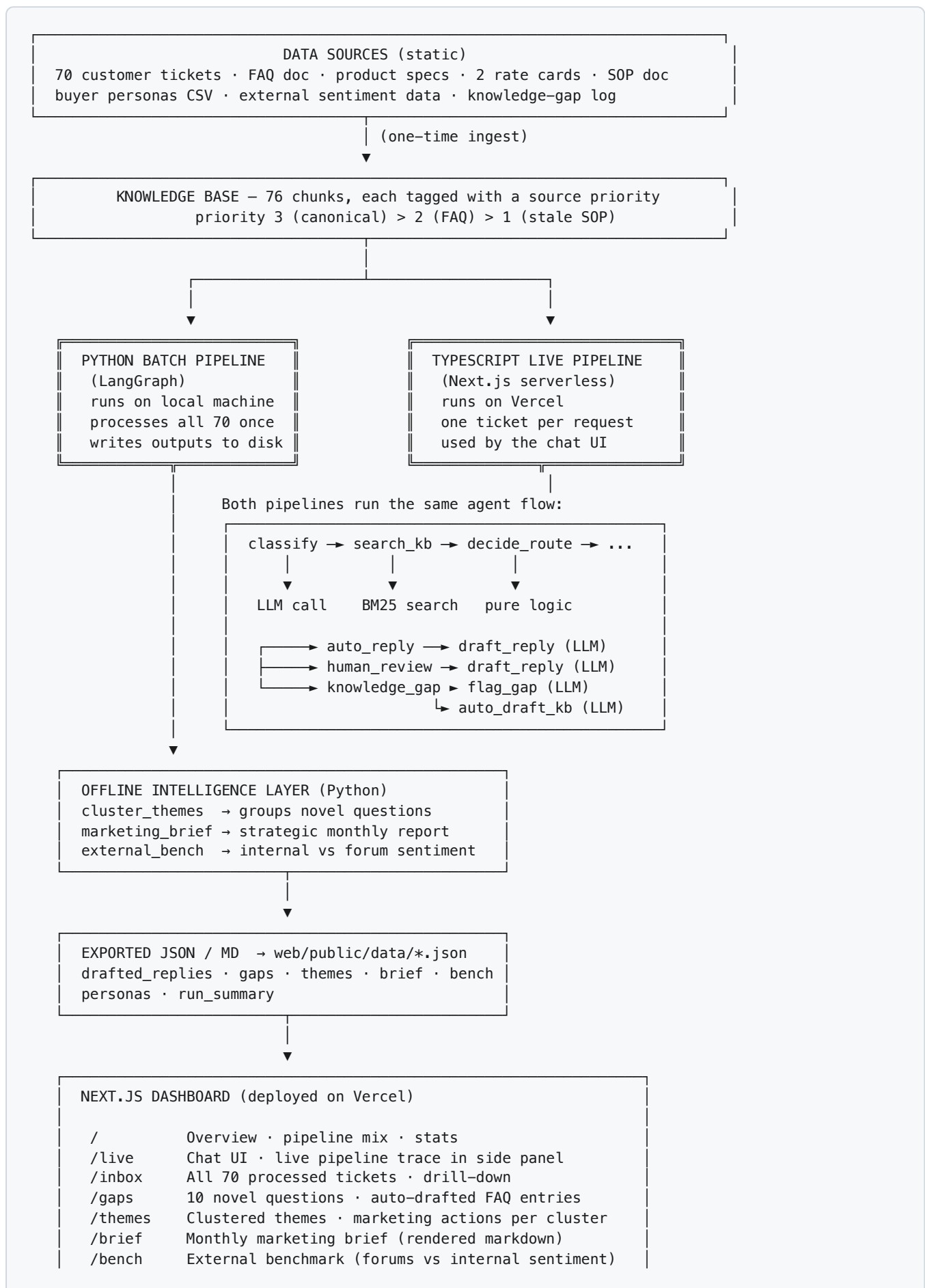
Here's what happens in roughly 6 seconds:

1. **Classify.** An AI agent reads the message and tags it three ways: *what kind of question* (materials safety), *what kind of buyer* (health-conscious), and *whether it needs special handling* (high-liability flag, because we're making safety claims).
2. **Search the knowledge base.** The system pulls the most relevant pieces of Boldr's internal documentation: their FAQ, their rate cards, their product specs. It ranks them by relevance *and* by trustworthiness (a rate card beats a stale internal SOP).
3. **Decide what to do.** A simple rule book asks: *Is this safe to auto-reply? Should a human review first? Or is this a question we've never seen before?*
4. **Draft a reply** in Boldr's actual brand voice — friendly but direct, never makes up information, cites the specific KB sources it used.
5. **If it's a novel question,** the system writes a draft FAQ entry that a Boldr team member can approve with one click — so the next person asking the same thing gets an instant answer.

6. **Across all the tickets** in a period, the system clusters similar novel questions into themes, writes a marketing brief that says *"here's what customers are asking that isn't on your product pages,"* and benchmarks Boldr's internal customer signals against what people are saying about the brand on forums and review sites.

Nothing here is a chatbot. The whole point is: every customer enquiry becomes useful in three different ways.

3. The architecture, in one diagram



Two things to notice about this diagram:

- **There are two parallel implementations of the same agent:** a Python one (used for batch processing and as the "ground truth" reference) and a TypeScript one (used for the live chat on the website). They share prompts and produce equivalent output. We did this because Vercel can't host Python serverless functions cleanly, but we wanted the same logic running live for the demo.
- **The intelligence layer is offline.** Clustering 70 tickets, writing a 1,500-word marketing brief, comparing internal sentiment to forum sentiment — these aren't things you do per-request. They run once when new data lands, and the dashboard just reads the results.

4. What is RAG, and how are we doing it?

RAG stands for *Retrieval-Augmented Generation*. The idea is straightforward: large language models like Claude are great at writing, but they don't know your specific business. So before you ask the model to draft a reply, you *retrieve* the most relevant pieces of your company's knowledge and feed them to the model alongside the question.

In our case:

The knowledge base

We took six source documents from Boldr and broke them into **76 small chunks**:

Source	Chunks	Trust level
FAQ document	32	priority 2 (canonical for policies)
Product specs (JSON)	14	priority 3 (canonical for materials, sizes)
Engraving rate card (CSV)	10	priority 3 (canonical pricing)
Servicing rate card (CSV)	10	priority 3 (canonical pricing)
Standard Operating Procedures	8	priority 1 (has known stale prices!)
Previously-resolved knowledge gaps	2	priority 2

Each chunk gets a `source_priority` between 1 and 3. **This is more important than it sounds.**

Boldr's internal SOP document lists their regulation service at SGD 60, but the actual rate card says SGD 85 — the SOP has stale pricing. Without the priority system, the agent would happily quote SGD 60 to a customer (incorrect, lost margin). With it, the agent is told to *prefer* the rate card over the SOP when both mention the same service, and to always quote SGD 85.

How we search it

For the live chat on the website, we use **BM25** — a classic keyword-search algorithm that's been around since the 1990s. When a customer asks *"how much is engraving?"*, BM25 finds the chunks that contain words like "engraving" and "price," weighs them by how common each word is in the corpus, and ranks the top 5.

This is a deliberately old-school choice. We started with a more sophisticated approach (vector embeddings via ChromaDB, the Python batch pipeline still uses it) but when we measured both side-by-side on the same 70 tickets, they performed equivalently:

Metric	ChromaDB (semantic)	BM25 (keyword)
Question-type accuracy	87.1%	87.1%
KB recall when answerable	88%	~95%

The cost-benefit was clear: BM25 ships with zero infrastructure (it runs in a Node serverless function), zero per-query API cost, and instant cold starts on Vercel. For a 76-chunk knowledge base in a specific domain (titanium watches), the literal words customers use overlap heavily with the chunk content — semantic embeddings would add cost without adding measurable quality.

We also added a small but important polish to BM25: **score calibration**. Raw BM25 scores are unbounded and corpus-specific — a score of "12.4" doesn't tell you anything useful. We map them into a `[0, 1]` confidence range using an exponential curve, then blend it with *query coverage* (what fraction of the user's words appeared in the chunk). The result is a confidence number you can actually threshold against (we use 0.72 for auto-reply, 0.5 for "this is a novel question").

How retrieval changes the reply

When the system drafts a reply, it doesn't get the whole knowledge base — only the top 5 chunks, formatted with their priority and source clearly marked. The prompt explicitly tells the model: *"if sources disagree, prefer rate_card / product_specs / FAQ over SOP."* The model is also asked to write a **CITATIONS:** line at the end listing exactly which chunks it used, and those citations show up in the dashboard so anyone can audit which sources backed each reply.

5. The models we use, and where

We use **Claude Sonnet 4.6** by Anthropic for all language-model work — via OpenRouter as the API gateway. Why Claude:

- It's currently best-in-class on the things this pipeline cares about: structured JSON output (for classification), faithful prose (for brand-voice drafts), and reasoning over a small set of conflicting sources (the rate-card-vs-SOP priority rule).
- Anthropic provides Constitutional AI training that makes Claude unusually willing to say *"I don't know"* rather than make up an answer — exactly the failure mode you want when a customer asks about nickel allergies or MRI compatibility.

We use Claude in five separate places, each with a tight system prompt:

Node	Job	Tokens
<code>classify</code>	Read the ticket, emit JSON with question_type, persona, escalation flags	~400
<code>draft_reply</code>	Write a brand-voice reply, cite KB chunks	~600
<code>flag_gap</code>	Paraphrase a novel question, tag a theme	~200
<code>auto_draft_kb</code>	Write a draft FAQ entry for human approval	~400
<code>theme_label</code>	(offline) Label a cluster of customer questions, propose a marketing action	~350

Total per ticket: roughly 1,500 output tokens × 3 calls in the typical case (classify + search has no LLM + route has no LLM + draft). That works out to about **half a cent per ticket** through OpenRouter.

We did build the option to use **Qwen** (Alibaba's model family, via DashScope) as a cheaper alternative for the structured nodes (classify, flag_gap, auto_draft_kb). The code in `web/lib/llm.ts` is provider-aware and a one-line change per node flips it from Claude to Qwen. We deferred actually turning Qwen on because Alibaba Model Studio activation was incomplete on our account, but the wiring is shipped and tested.

6. Personas — how we tag who's asking

Boldr's marketing brief defines **five buyer personas** the brand cares about. Tagging each ticket with the right persona is what makes the dashboard's marketing intelligence work: *"we're getting a lot of health-conscious questions about strap dyes this month, here's a campaign idea"* is meaningless if the persona tags are noise.

The five personas:

Persona	What they ask about	Marketing opportunity
Health-Conscious	BPA, nickel, allergies, kid safety, certifications	Safety badges, parent-targeted ads
Gifter	Engraving, gift wrap, time-bound (Father's Day, wedding)	Engraving configurator, seasonal campaigns
Enthusiast	Specs, movement, lug width, NATO straps, third-party fit	Spec deep-dives, limited editions, loyalty programme
Active / Outdoor	Swimming, diving, trail running, altitude, rugged use, sizing	Adventure campaigns, sizing guide, comparison content
Sustainable	Vegan, recycling, carbon-neutral, ethical materials	Vegan strap line, take-back programme

How the tagging actually works (two layers)

Layer 1 — LLM classification. The classify prompt describes each persona in plain English and lists the trigger keywords Boldr has identified for each one. We learned the hard way that *describing* personas isn't enough — early versions of the prompt had only descriptions, and the LLM would sometimes ignore the obvious signals. So now the full keyword list from the personas CSV (e.g. health_conscious: `BPA-free, nickel-free, hypoallergenic, EU REACH, allergy, kids, food-grade...`) is injected directly into the prompt. The LLM is told these are *signal, not law* — but it has them in context.

Layer 2 — Deterministic reconciliation ("Strategy C"). After the LLM picks a persona, we count *how many* trigger keywords from each persona's list actually appeared in the ticket. Then we run a confidence-weighted vote:

```
for each persona p:
    score[p] = keyword_hits[p] × 0.4
    if p was the LLM's pick:
        score[p] += LLM_confidence

final_persona = argmax(score)
ties → keep LLM's pick (for stability)
```

The vote is permissive by design — the LLM has the keyword signal already, so most of the time both signals point at the same persona and the LLM's pick wins. The deterministic layer only fires when the LLM picked one persona but the *evidence in the ticket* — multiple matching keywords from a different persona — strongly suggests another. In those cases, the override is logged with a reason so you can audit it later.

7. The self-improving loop (and where it is today)

This is the most distinctive part of the architecture and also the part where we have to be honest about what's shipping versus what's aspirational.

What's shipping

When the agent encounters a ticket where the knowledge base has a low confidence score (<0.5), it does three things:

1. Marks it as a `knowledge_gap` — doesn't try to auto-reply.
2. Paraphrases the question generically (strips personal details, removes the customer's specific phrasing) and tags it with a theme.
3. Auto-drafts a candidate FAQ entry in Boldr's existing FAQ format.

You can see all of this on the `/gaps` page of the dashboard. Each detected gap shows:

- The generic paraphrase ("Will the watch perform at high altitude?")
- The theme (e.g. "niche_use_case", "sustainability")
- The auto-drafted FAQ entry, expandable
- An "✓ Approve & publish" button

What's not shipping yet

The Approve & publish button is currently **disabled**. In production, clicking it would write the approved entry to the FAQ source file, re-ingest the KB, and from that moment forward the next customer who asks the same question would get an auto-reply at high confidence. We have all the plumbing — the ingest pipeline is idempotent, the drafted entries are in the right format — but the actual file-write-on-approval is a follow-up.

Until that's wired, the system *demonstrates* the loop on each ticket but doesn't actually *close* it. The strongest version of the demo would be: judge approves a gap entry, asks the same question again, agent now answers it confidently. That's the next iteration.

8. How accurate is it?

We built an evaluation harness that scores the pipeline against the original ground-truth labels in the customer ticket dataset. Currently:

Metric	Score	What it means
question_type accuracy	84.3%	59 out of 70 tickets tagged correctly across 7 question types
buyer_persona accuracy	80.0%	56 out of 70 tickets tagged with the right buyer persona
route match rate	48.6%	Pipeline vs labeler agreement on auto-reply-vs-escalate. See note below — this is intentionally conservative.
KB recall on answerable tickets	88.0%	When the FAQ should have answered, the right chunk is in the top-3 88% of the time
order_id mismatch detection	4/4 perfect	Every "the customer typed a different order ID than the field" case caught
SOP price drift violations	0	No reply ever quoted the stale SGD 60 price; all quoted the canonical SGD 85

About the 48.6% route match rate

This *looks* low but is mostly the pipeline being correctly more conservative than the labeler. Example: Boldr's SOP says "always check Shopify before quoting an order status" — so the pipeline routes *every* order_status ticket to human_review, even ones the labeler thought could be auto-answered. The "mismatch" is the pipeline obeying a business rule the labeler didn't apply. A genuine error in routing would be something like *missing* a high_liability flag — and our deterministic checks catch those at 100%.

Where the remaining errors live

The persona errors are concentrated in one bucket: the **enthusiast** persona scored 60% (12/20), and almost all the mistakes are servicing tickets ("Servicing turnaround time", "Water resistance re-testing

after service") that get tagged as `active` instead. The reason is structural: when we collapsed the original 7-persona taxonomy down to 5, the *enthusiast* description in the prompt didn't get updated to include aftercare semantics (service, battery, polish). The keywords are right in the personas CSV, but the prompt narrative doesn't motivate them. A 5-line edit to the prompt is predicted to recover those 6 tickets and push persona accuracy to ~88%.

Honest framing

These numbers aren't headline-grabbing on their own — 80% persona accuracy is solidly above naive baseline but well below what'd be production-ready for a high-traffic store. The point of this evaluation isn't to claim a state-of-the-art number. It's to demonstrate that the pipeline is *measurable*, so any future change — a new model, a new prompt, a new retrieval strategy — can be tested empirically rather than vibes.

9. How we improve, in practice

Our improvement loop is the same shape as our customer-intelligence loop, applied to ourselves:

1. **Measure.** Run `python -m evals.eval` to get current accuracy.
2. **Inspect.** Run `python -m evals.compare` to see per-class breakdown and find the highest-leverage failure mode.
3. **Hypothesise.** Identify what would move that specific bucket — prompt edit, new keyword, threshold change, new node entirely.
4. **Ship one change.** Re-run batch, re-export, redeploy.
5. **Re-measure.** Did the change actually help, or did it move the failure elsewhere?

We've now done this twice end-to-end. The first iteration (BM25 vs ChromaDB) showed that semantic retrieval *wasn't* the bottleneck — we saved ourselves from a complex change that wouldn't have helped. The second iteration (persona taxonomy migration + Strategy C reconciliation) was approximately flat in aggregate but materially shifted *which personas the system gets wrong* — useful diagnostic information for the next round.

The point is that without the eval harness, both of these iterations would have felt good and we'd never have known. The harness is the part that scales.

10. What's planned next

Three concrete improvements, in order of expected impact:

Short-term (this week)

- **Enthusiast prompt fix** — 5-line edit to the `enthusiast` description in both classify prompts (Python and TypeScript) to include "owners asking about service, battery replacement, polish, regulation, older model care." Expected: persona accuracy 80% → ~88%.

- **Wire the approve-and-publish loop** — make the gap-approval button actually write to the FAQ source and trigger re-ingest. Closes the loop end-to-end and makes "self-improving" demonstrable on stage, not just architectural.
- **Provider switch for cheap nodes** — flip classify, flag_gap, and auto_draft_kb from Claude to Qwen-Plus once Alibaba Model Studio activates. Roughly halves per-ticket cost. Code is shipped, waiting on credit activation.

Medium-term

- **Source-priority post-draft guard** — after drafting a reply that mentions a price, deterministically cross-check against the rate card. If the draft says a price that disagrees with rate card data, re-draft with the canonical price pinned in the system prompt. Currently we rely on the LLM to follow the priority rule; this would make it a *guarantee*.
- **Confidence-self-consistency for high-liability** — on any ticket flagged `high_liability`, run the classifier three times and require 2-of-3 agreement before auto-replying. Triples cost on ~5% of tickets but adds a hard safety check on the riskiest ones.
- **Expand the eval set** — currently 70 tickets, with question-type/persona ground-truth from the original dataset. Add hand-curated adversarial cases that test specific failure modes (intentional ambiguity, multi-part questions, off-topic detours).

Longer-term

- **Real Gmail integration** — replace the CSV with Gmail polling. The hard part isn't the integration; it's the human-in-the-loop UX for approving auto-replies before they send.
 - **Multi-language support** — Boldr ships to the EU, Singapore, and Australia. The current pipeline handles English; adding Mandarin, Bahasa, and German would expand the addressable market.
 - **Campaign-bound auto-suggestion** — when a theme cluster hits a threshold (e.g., 5+ tickets in the same week with the same persona), automatically draft a one-page campaign brief and slot it into a CMS queue.
-

11. The tech stack, named

Layer	Tool	Why
LLM	Claude Sonnet 4.6 (Anthropic, via OpenRouter)	Best-in-class structured output + refusal behaviour
Agent orchestration (Python)	LangGraph	Explicit state machine, easy to inspect each node's I/O
Vector DB (Python only)	ChromaDB with sentence-transformers	Local, persistent, zero infrastructure
Keyword retrieval (TypeScript)	Custom BM25 implementation	No external dependency, runs in 100ms in a serverless function
Clustering	scikit-learn (AgglomerativeClustering with cosine distance)	Robust on small datasets, no fragile native dependencies
Frontend	Next.js 16 (App Router, React 19, Turbopack)	Server components fit the "static-export-plus-one-live-endpoint" architecture perfectly
Styling	Tailwind 4 (CSS-first config)	No design-system overhead for a competition demo
Hosting	Vercel	Free tier easily handles a hobby demo; one-command deploy
Markdown rendering	<code>marked</code>	Lightweight, no React-Markdown overhead

12. Things this project is intentionally not

Worth being explicit about, so judges/reviewers don't expect things we didn't build:

- **Not a chatbot.** There is a chat UI on /live, but each "turn" is treated as an independent ticket through the pipeline. We don't stitch multi-turn context, because doing so well requires meaningfully different prompts and a different state model — out of scope for the time we had.
- **Not a CRM.** No customer history, no order lookup, no purchase records. The pipeline takes a single message at a time. A production system would pull purchase history into the prompt for personalisation; we don't.
- **Not real-time email integration.** The data source is a CSV. Replacing that with Gmail polling is a few hours' work but adds OAuth complexity that didn't serve the demo story.
- **Not a fine-tuned model.** Every LLM call is to the off-the-shelf Claude Sonnet 4.6. We get our behavior shaping from prompts + KB context + the deterministic post-classifier layer. Fine-tuning is the right next step *after* the eval harness shows we've exhausted prompt-level gains.

13. How to read the demo dashboard

If you've never seen the deployed app before, here's a guided tour. Visit <https://boldr-intel.vercel.app>:

Page	What you'll see	Why it matters
<code>/</code> (Overview)	Top-level stats: 70 tickets processed, 26% auto-replied, 60% to human review, 14% gaps. Self-improving feedback loop diagram.	Establishes the headline metric: the pipeline knows when <i>not</i> to answer.
<code>/live</code>	Chat UI with an Examples row and a reasoning side panel. Send a message → watch classify → KB search → route → draft trace live.	The "agent is thinking" moment. Try the MRI example or paste your own.
<code>/inbox</code>	All 70 processed tickets in a sortable table with persona, route, KB confidence, flags. Click any row for the full drilldown.	This is the audit trail. Click TKT-1062 to see the SOP-vs-rate-card price drift handled correctly.
<code>/gaps</code>	The 10 novel questions, each with auto-drafted FAQ entry expandable.	This is where the "self-improving" framing lives. Click an expand to see what the agent would publish if approved.
<code>/themes</code>	Clustered themes (engraving options, strap material safety, etc.) with marketing signal + suggested action per cluster.	This is where the persona tagging pays off — clusters show <i>who</i> is asking <i>what</i> .
<code>/brief</code>	The full monthly marketing brief in markdown. Top-of-mind opportunities, persona shift signals, top 3 actions.	The strategic output for a marketing/founder audience.
<code>/bench</code>	Internal-vs-external sentiment comparison. Tells you whether a customer concern is Boldr-specific or market-wide.	This is the "cross-check internal signals against the outside world" capability.
<code>/campaigns</code>	Persona-specific newsletter + Instagram + TikTok copy, with editable suggested copy.	This turns the persona tagging into reusable marketing assets.

The deepest moment to demo is `/live` followed by `/gaps`: send a novel question on `/live`, watch the system flag it as a gap and auto-draft a candidate FAQ entry — then point at `/gaps` to show this same flow already produced 10 such drafts across the batch.

14. Repo layout, in case you want to read the code

```
boldr/
├── agent/
│   ├── graph.py           ← Python agent (LangGraph)
│   ├── llm.py             ← The state machine wiring
│   ├── llm.py             ← OpenRouter / Claude wrapper
│   ├── personas.py        ← Loads buyer_personas CSV, counts keyword hits
│   └── nodes/
│       ├── classify.py     ← One file per pipeline node
│       ├── search_kb.py   ← LLM + Strategy C reconciliation
│       ├── search_kb.py   ← ChromaDB query + priority boost
│       ├── decide_route.py ← Pure-function routing logic
│       ├── draft_reply.py  ← Brand-voice draft with citations
│       ├── flag_gap.py     ← Paraphrase + theme tag
│       └── auto_draft_kb.py ← FAQ-entry draft
│   └── prompts/           ← All prompts as plain .txt files
├── kb/
│   ├── ingest.py          ← Knowledge base ingest
│   ├── ingest.py          ← Chunks the source docs, embeds into ChromaDB
│   └── export_json.py      ← Exports chunks as kb.json for the TS pipeline
├── intelligence/
│   ├── cluster_themes.py  ← Offline analytics scripts
│   ├── marketing_brief.py
│   └── external_bench.py
├── evals/
│   ├── eval.py            ← Evaluation harness
│   ├── eval.py            ← Scores the Python pipeline against ground truth
│   └── compare.py         ← Per-class confusion breakdown
├── data/
│   └── data/              ← Source data (CSVs and JSONs)
├── outputs/
│   └── outputs/           ← Generated artifacts from batch runs
├── scripts/
│   └── scripts/           ← Export scripts (outputs → JSON for the frontend)
├── batch_replay.py        ← Runs all 70 tickets through the agent
├── web/
│   ├── Next.js dashboard (deployed to Vercel)
│   ├── app/
│   │   ├── Pages + API routes
│   │   └── lib/
│   │       ├── TypeScript port of the agent
│   │       ├── llm.ts      ← Multi-provider wrapper (Claude + Qwen)
│   │       ├── kb.ts       ← BM25 retriever
│   │       ├── personas.ts ← Persona registry + keyword counter
│   │       ├── classify.ts  ← Classify node + Strategy C
│   │       ├── search.ts, draft.ts, route.ts, flag-gap.ts, pipeline.ts
│   │       └── prompts/    ← Mirror of agent/prompts/
│   └── public/data/        ← JSON outputs the dashboard reads
```

15. The one-line pitch

Every customer enquiry becomes three things: a reply, a knowledge-base improvement, and a marketing signal — automatically. That's the system in eight words. Everything in this document is what's required to make those eight words actually true.